

Competitive Programming

Section 16: DFS, Components, And Cycles

Competitive Programming Course

2026-07-23

Table of contents

1	Why DFS after BFS	3
2	Recognition	4
3	Vocabulary	5
3.1	Depth-first search	5
3.2	Parent	5
3.3	Preorder and postorder	5
4	Recursive DFS primer	7
5	Iterative DFS primer	8
5.1	Pattern 1: record order, then reverse it	8
5.2	Pattern 2: stack frames with next-neighbor index	9
6	Worked example 1: subtree sizes	10
7	Worked example 2: undirected cycle detection	11
8	Worked example 3: directed cycle detection	12
9	DFS colors	13
10	Components and disconnected graphs	14
11	Complexity	15
12	Implementation notes	16
12.1	C++	16
12.2	Python	16
13	Local exercises	17
13.1	A. Subtree Sizes	17
13.2	B. Undirected Cycle	17
13.3	C. Directed Cycle	17
14	Submission checklist	18
15	Exercises	19

1 Why DFS after BFS

BFS is about layers. It is the right mental model for shortest paths in unweighted graphs.

DFS is about going deep, finishing a branch, and coming back with information. That makes it the natural tool for:

```
tree subtree sizes  
parent and child relationships  
entry/exit reasoning  
cycle detection  
postorder dynamic programming on trees
```

This section does not assume you already know DFS. It teaches the traversal, the state variables, and the exact invariants used in the local exercises.

2 Recognition

Look for these statement signals:

Statement signal	Likely DFS idea
rooted tree, descendants, subtree	DFS with parent and postorder
“does this graph contain a cycle?”	DFS with parent/colors
need to process children before parent	DFS postorder
connected component traversal, no shortest path	BFS or DFS both work
dependency graph has impossible circular dependency	directed DFS colors

BFS and DFS often visit the same set of vertices. The difference is the order and what that order makes easy.

3 Vocabulary

3.1 Depth-first search

Depth-first search starts from a vertex, chooses an unvisited neighbor, goes as deep as possible, and only then backs up.

For this graph:

```
1 -- 2 -- 4
|    |
3    5
```

one possible DFS from 1 is:

```
enter 1
  enter 2
    enter 4
    exit 4
    enter 5
    exit 5
  exit 2
  enter 3
  exit 3
exit 1
```

The exact order depends on adjacency-list order. Correctness should not depend on one accidental ordering unless the problem explicitly asks for it.

3.2 Parent

When DFS moves from u to an unseen neighbor v , we can record:

```
parent[v] = u
```

In an undirected graph, **parent** prevents us from immediately treating the edge back to the previous vertex as a cycle.

3.3 Preorder and postorder

DFS has two important moments for each vertex:

```
preorder:  when we first enter the vertex  
postorder: after all children have been processed
```

Subtree sizes need postorder:

```
size[u] = 1 + sum(size[child] for child of u)
```

You cannot finish `size[u]` until the child sizes are known.

4 Recursive DFS primer

The most direct DFS is recursive:

```
void dfs(int u, int p) {
    parent[u] = p;
    for (int v : adj[u]) {
        if (v == p) continue;
        dfs(v, u);
    }
}
```

Python:

```
def dfs(u, p):
    parent[u] = p
    for v in adj[u]:
        if v == p:
            continue
        dfs(v, u)
```

This mirrors the definition. The function call stack remembers where to return after a child finishes.

Contest warning: recursive DFS can overflow when the graph is a long chain of 200000 vertices. C++ may stack overflow. Python needs `sys.setrecursionlimit`, and even then recursion can be fragile. This course uses iterative solutions in the reference code when depth can be large.

5 Iterative DFS primer

An explicit stack can simulate recursive DFS.

For reachability:

```
push start
mark start

while stack not empty:
    u = pop
    for v in adj[u]:
        if v is unseen:
            mark v
            push v
```

This visits the component, but it does not directly give postorder. For postorder, use one of two patterns.

5.1 Pattern 1: record order, then reverse it

This is enough on a tree:

```
parent[root] = root
stack = [root]
order = []

while stack not empty:
    u = pop
    append u to order
    for v in adj[u]:
        if v == parent[u]: continue
        parent[v] = u
        push v

for u in reversed(order):
    add size[u] to size[parent[u]]
```

Because every child is pushed after its parent is recorded, reversing the discovery order processes descendants before ancestors.

5.2 Pattern 2: stack frames with next-neighbor index

This more closely simulates recursion and is useful for directed cycle detection:

```
mark start as discovered
push (start, 0)

while stack is not empty:
    u, i = top frame

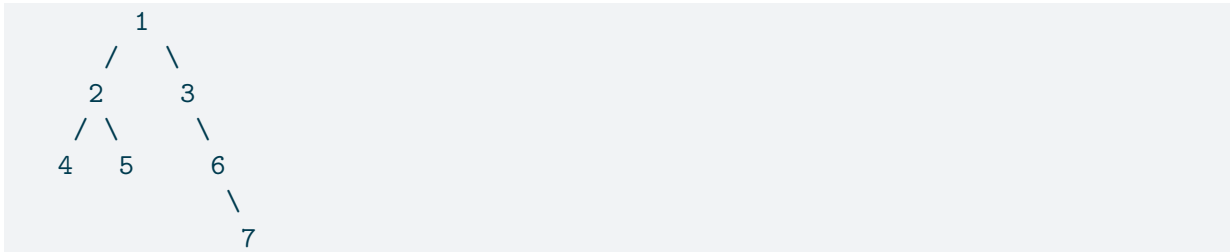
    if i == number of neighbors of u:
        finish u
        pop the top frame
    else:
        v = adj[u][i]
        increment top frame's index

        if v is not discovered:
            mark v as discovered
            push (v, 0)
```

The push of $(v, 0)$ is the iterative equivalent of calling `dfs(v)`. The frame for u remains directly below it, with its index already advanced, so after v finishes the search resumes at u 's next neighbor. A vertex is active in the DFS exactly while its frame remains on this stack.

6 Worked example 1: subtree sizes

Tree rooted at 1:



Leaves have subtree size 1:

```
size[4] = 1  
size[5] = 1  
size[7] = 1
```

Then:

```
size[6] = 1 + size[7] = 2  
size[2] = 1 + size[4] + size[5] = 3  
size[3] = 1 + size[6] = 3  
size[1] = 1 + size[2] + size[3] = 7
```

The important part is the direction of computation:

```
children first, parent last
```

7 Worked example 2: undirected cycle detection

Graph:

```
1 -- 2 -- 3
|       |
+-----+
```

DFS:

```
enter 1, parent none
enter 2, parent 1
enter 3, parent 2
see neighbor 1
```

At vertex 3, neighbor 1 is already seen and is not `parent[3]`. That means there is another route back to an earlier vertex:

```
1 -> 2 -> 3 -> 1
```

So the graph has a cycle.

In an undirected graph, seeing the parent is normal:

```
2 sees neighbor 1, but 1 is parent[2], so ignore it
```

8 Worked example 3: directed cycle detection

Directed graph:

```
1 -> 2 -> 3 -> 4
      ^         |
      +-----+
```

DFS colors:

```
0 = unvisited
1 = currently active in this DFS stack
2 = completely finished
```

Trace:

```
enter 1: color[1] = 1
enter 2: color[2] = 1
enter 3: color[3] = 1
enter 4: color[4] = 1
4 sees 2, and color[2] = 1
```

An edge to an active vertex is a back edge. It forms a directed cycle.

An edge to a finished vertex is different:

```
u -> v and color[v] = 2
```

That edge does not prove a cycle, because v is not an ancestor on the current DFS path.

9 DFS colors

For directed cycle detection, use three states:

Color	Meaning
0	never entered
1	entered but not finished; active on DFS stack
2	finished; all outgoing edges already checked

Cycle rule:

```
During DFS, if we inspect edge u -> v and color[v] == 1,  
then a directed cycle exists.
```

Why the same color trick is not needed for undirected simple graphs:

```
undirected cycle detection only needs seen[v] and parent skipping
```

10 Components and disconnected graphs

Many graph inputs are disconnected. DFS from vertex 1 may not see everything.

The standard outer loop is:

```
for start in all vertices:
    if start is unvisited:
        start a new DFS
```

This matters for cycle detection:

```
one component may be acyclic
another component may contain the cycle
```

11 Complexity

With adjacency lists:

```
Time:  $O(V + E)$   
Memory:  $O(V + E)$ 
```

Each vertex is entered a constant number of times, and each adjacency-list item is inspected a constant number of times.

For a tree:

```
 $E = V - 1$   
Time:  $O(V)$ 
```

12 Implementation notes

12.1 C++

Use adjacency lists:

```
vector<vector<int>>> adj(n);
```

For iterative DFS stacks:

```
vector<int> st;  
vector<pair<int,int>> frames;
```

Use `vector<char>` for Boolean or color arrays:

```
vector<char> color(n, 0);
```

12.2 Python

Use lists as stacks:

```
stack = [start]  
u = stack.pop()  
stack.append(v)
```

This is efficient at the end of the list. Do not use `pop(0)` for stacks or queues.

For large input:

```
data = list(map(int, sys.stdin.buffer.read().split()))
```

13 Local exercises

13.1 A. Subtree Sizes

Use DFS to root the tree at 1, then compute:

```
subtree[u] = 1 + sum(subtree[child])
```

The reference solution uses discovery order reversed to get postorder.

13.2 B. Undirected Cycle

Use DFS over every component. For each edge from u to v :

```
if v == parent[u]: ignore it
else if v is already seen: cycle exists
else visit v with parent u
```

The input is simple, so there are no parallel-edge corner cases.

13.3 C. Directed Cycle

Use DFS colors:

```
0 unvisited
1 active
2 finished
```

An edge to color 1 means **CYCLIC**.

14 Submission checklist

Before submitting:

```
Did I run DFS from every component when the graph may be disconnected?  
Did I store undirected edges both ways?  
Did I skip only the parent edge in undirected cycle detection?  
For directed graphs, did I distinguish active and finished vertices?  
For subtree sizes, did I process children before parents?  
Did I avoid recursive depth issues on large chains?
```

15 Exercises

Use A to practice entry/exit information in a rooted DFS, B to distinguish a parent edge from a cycle, and C to use three directed-DFS colors. Test a single vertex, a self-contained cycle, and disconnected input.